

Wynns ρ -Algorithmus

Lucas Posern

Juli 2025

Inhaltsverzeichnis

1	Einführung	1
2	Notwendigkeit des ρ-Algorithmus	1
3	Wynns ρ-Algorithmus	2
3.1	Idee	2
3.2	Definition	2
3.3	Beispiel	3
3.4	Beschleunigung	5
3.5	Schwächen	9
4	Beispiele, Tests & Vergleiche mit anderen Verfahren	11
4.1	Linear konvergente Folgen: Vergleich von ρ und Aitken- Δ^2 . .	11
4.2	Alternierende Folgen: Vergleich von ρ und ϵ	12
4.3	Logarithmisch konvergente Folgen	13
4.4	Langsam konvergente Folgen	14
4.5	Modifikationen	15
4.5.1	Modifikation 1 - $\rho(\gamma)$ -Algorithmus	15
4.5.2	Automatisierter $\rho(\gamma)$ -Algorithmus	17
4.5.3	Modifikation 2 - $\rho(\gamma, p)$ -Algorithmus	18
	Literaturverzeichnis	20

1 Einführung

Allgemeine Aussagen zu Konvergenzproblemen und deren Beschleunigungen entnehmen wir aus [Weniger, 1989, S. 194–199]. In vielen Bereichen der numerischen Mathematik und der wissenschaftlichen Berechnung spielen iterative Verfahren eine zentrale Rolle. Sie dienen dazu, Lösungen von Gleichungen, Reihen oder Fixpunktproblemen Schritt für Schritt zu approximieren. Ein häufiges Problem solcher Verfahren ist jedoch die oft langsame Konvergenz, die den Rechenaufwand erheblich erhöht und in der Praxis die Effizienz und Anwendbarkeit einschränkt. Hier setzt die Konvergenzbeschleunigung an. Sie umfasst Methoden, die darauf abzielen, die Geschwindigkeit der Annäherung an den Grenzwert deutlich zu verbessern.

Wynns ρ -Algorithmus, benannt nach Peter Wynn, der ihn in den 1950er Jahren entwickelte, ist eine dieser Methoden zur Konvergenzbeschleunigung. Der Algorithmus entstand ursprünglich aus der Herausforderung heraus, um die Handhabung langsam konvergierender oder gar divergierender Reihen und Folgen zu verbessern. In der Praxis zeigte sich, dass viele klassische Reihenentwicklungen und iterative Verfahren oft erst nach einer sehr großen Anzahl von Schritten brauchbare Ergebnisse liefern, sofern sie überhaupt in endlicher Zeit konvergieren. Der ρ -Algorithmus bietet hier eine elegante Möglichkeit, durch rekursive Transformationen die Konvergenzrate signifikant zu erhöhen.

Der Wynnsche ρ -Algorithmus ist hierbei eine Erweiterung des Wynnschen ϵ -Algorithmus. Sie unterscheiden sich zwar nur geringfügig, sind aber dadurch für verschiedene Konvergenzklassen anwendbar.

Im weiteren Verlauf dieser Arbeit wird der Wynnsche ρ -Algorithmus detailliert vorgestellt, seine mathematischen Grundlagen erläutert sowie typische Anwendungsbeispiele gezeigt, die seine Bedeutung und Leistungsfähigkeit im Bereich der Konvergenzbeschleunigung verdeutlichen.

2 Notwendigkeit des ρ -Algorithmus

Für Aussagen zum Wynn- ϵ -Algorithmus beziehen wir uns auf [Wynn, 1966].

Der ϵ -Algorithmus, eingeführt von Peter Wynn (1956), machte einen wesentlichen Fortschritt für die Konvergenzbeschleunigung durch seine elegante Rekursion:

$$\epsilon_{k+1}^{(n)} = \epsilon_{k-1}^{(n+1)} + \frac{1}{\epsilon_k^{(n+1)} - \epsilon_k^{(n)}}$$

Dennoch kam das Verfahren an Grenzen:

1. **Nicht immer erfolgreich:** Für logarithmische Konvergenz versagt der ϵ -Algorithmus.
2. **Parameterstarre:** Die feste Koeffizientenstruktur (+1 im Zähler) lässt keine Anpassung an unterschiedliche asymptotische Entwicklungen zu.

Der ρ -Algorithmus (Wynn, 1966) behebt diese Defizite durch eine fundamentale strukturelle Modifikation:

$$\rho_{k+1}^{(n)} = \rho_{k-1}^{(n+1)} + \frac{k+1}{\rho_k^{(n+1)} - \rho_k^{(n)}}$$

Seine Innovationskraft liegt in folgenden Kernaspekten:

1. **Beschleunigung für andere Folgen:** Dem ρ -Algorithmus gelingt die Konvergenzbeschleunigung für logarithmisch konvergente Folgen durch die dynamische Skalierung des Zählers mit $k+1$.
2. **Parametrisierbarkeit:** Die Erweiterung zum $\rho(\gamma, p)$ -Algorithmus ([Abschnitt 4.5.1](#)) ermöglicht die Anpassung an beliebige Entwicklungen der Form $s_n \sim s + n^\theta(c_0 + c_1/n + \dots)$.

3 Wynns ρ -Algorithmus

3.1 Idee

Der Wynnsche ρ -Algorithmus ist eine Technik zur Beschleunigung der Konvergenz von Zahlenfolgen. Insbesondere bei langsam konvergenten Folgen oder bei Reihen, deren Summen schwierig direkt zu berechnen sind, bietet der Algorithmus eine effektive Möglichkeit zur Verbesserung der numerischen Genauigkeit. Er gehört zu den sogenannten nichtlinearen Transformationen und ist insbesondere für logarithmisch konvergente Folgen geeignet.

Der Grundgedanke hinter Konvergenzbeschleunigungsverfahren ist, aus einer gegebenen Folge $\{s_n\}$ eine neue Folge $\{\rho_n\}$ zu erzeugen, die gegen denselben Grenzwert konvergiert, jedoch deutlich schneller.

3.2 Definition

Wir nutzen die Definition des ρ -Algorithmus aus [Weniger, 1989, S. 34–35]. Der ρ -Algorithmus basiert auf einer rekursiven Konstruktion einer Tabelle ähnlich dem ε -Algorithmus. Gegeben sei eine Folge $\{s_n\}_{n=0}^\infty$, so wird die Rho-Tabelle $\rho_n^{(k)}$ wie folgt rekursiv definiert:

$$\rho_{-1}^{(n)} = 0 \quad (3.1)$$

$$\rho_0^{(n)} = s_n \quad (3.2)$$

$$\rho_{k+1}^{(n)} = \rho_k^{(n+1)} + \frac{x_{n+k+1} - x_n}{\rho_k^{(n+1)} - \rho_k^{(n)}}, \quad k \geq 0 \quad (3.3)$$

Voraussetzung für die Berechnung ist, dass die Interpolationspunkte x_n eine aufsteigende divergente Folge bilden:

$$x_0 < x_1 < \dots$$

$$\lim_{n \rightarrow \infty} x_n = \infty$$

Im Fall, dass die Interpolationspunkte einfach der Nummer des Folgenglieds entsprechen ($x_n = n$) lässt sich der Algorithmus auch so definieren:

$$\rho_n^{(-1)} = 0 \quad (3.4)$$

$$\rho_n^{(0)} = s_n \quad (3.5)$$

$$\rho_n^{(k+1)} = \rho_{n+1}^{(k)} + \frac{k+1}{\rho_{n+1}^{(k)} - \rho_n^{(k)}}, \quad k \geq 0 \quad (3.6)$$

Der Wert $\rho_0^{(2k)}$ ergibt die transformierte Näherung an den Grenzwert. Die ungeraden Indizes $\rho_n^{(2k+1)}$ besitzen dabei nur indirekt eine Bedeutung für die Approximation.

3.3 Beispiel

Als einfaches Beispiel betrachten wir die einfache Folge

$$s_n = \frac{2 + 3x_n}{1 + x_n}, \quad x_n = n + 1$$

$$s = \lim_{n \rightarrow \infty} s_n = \lim_{n \rightarrow \infty} \frac{2 + 3(n+1)}{1 + (n+1)} = 3$$

Die Folgenglieder lauten:

$$s_0 = \frac{2 + 3x_n}{1 + x_n} = \frac{2 + 3(0+1)}{1 + (0+1)} = \frac{5}{2} = 2.5$$

$$s_1 = [\dots] = \frac{8}{3} \approx 2.667$$

$$s_2 = [\dots] = 2.75$$

$$s_3 = [\dots] = 2.8$$

ρ -Iterationen:

$$\begin{aligned}\rho_{-1}^{(n)} &= 0 \\ \rho_0^{(n)} &= s_n \\ \rho_{k+1}^{(n)} &= \rho_{k-1}^{(n+1)} + \frac{x_{n+k+1} - x_n}{\rho_k^{(n+1)} - \rho_k^{(n)}}\end{aligned}$$

Fülle die ρ -Tabelle:

n	$\rho_{-1}^{(n)}$	$\rho_0^{(n)}$
0	0	2,500
1	0	2,667
2	0	2,750
3	0	2,800

Stufe 1, k=0: $\rho_1^{(n)} = \rho_{-1}^{(n+1)} + \frac{x_{n+1} - x_n}{\rho_0^{(n+1)} - \rho_0^{(n)}}$

$$\begin{aligned}\rho_1^{(0)} &= 0 + \frac{x_1 - x_0}{s_1 - s_0} = \frac{2 - 1}{2,667 - 2,500} = \frac{1}{0,1667} \approx 6 \\ \rho_1^{(1)} &= 0 + \frac{x_2 - x_1}{s_2 - s_1} = \frac{3 - 2}{2,750 - 2,667} = \frac{1}{0,0833} \approx 12 \\ \rho_1^{(2)} &= 0 + \frac{x_3 - x_2}{s_3 - s_2} = \frac{4 - 3}{2,800 - 2,750} = \frac{1}{0,05} = 20\end{aligned}$$

Stufe 2, k=1: $\rho_2^{(n)} = \rho_0^{(n+1)} + \frac{x_{n+1+1} - x_n}{\rho_1^{(n+1)} - \rho_1^{(n)}}$

$$\begin{aligned}\rho_2^{(0)} &= s_1 + \frac{x_2 - x_0}{\rho_1^{(1)} - \rho_1^{(0)}} = 2,667 + \frac{3 - 1}{12 - 6} = 2,667 + \frac{2}{6} \approx 3 \\ \rho_2^{(1)} &= s_2 + \frac{x_3 - x_1}{\rho_1^{(2)} - \rho_1^{(1)}} = 2,750 + \frac{4 - 2}{20 - 12} = 2,750 + \frac{2}{8} = 3\end{aligned}$$

ρ -Tabelle jetzt:

$\Rightarrow$ Die Beschleunigung war erfolgreich.

Die transformierten Werte $\rho_2^{(0)}$ und $\rho_2^{(1)}$ entsprechen dem Grenzwert $s = 3$. Das ist eine starke Verbesserung in der Beschleunigung der Konvergenz.

n	$\rho_{-1}^{(n)}$	$\rho_0^{(n)}$	$\rho_1^{(n)}$	$\rho_2^{(n)}$
0	0	2,500	6	3
1	0	2,667	12	3
2	0	2,750	12	
3	0	2,800		

3.4 Beschleunigung

Den Beweis zur Beschleunigung einer Folge durch den ρ -Algorithmus nehmen wir aus [Osada, 1996, S. 522–525].

In dieser Arbeit beweisen wir, dass der ρ -Algorithmus die Konvergenz einer Folge

$$s_n \sim s + n^\theta \left(c_0 + \frac{c_1}{n} + \frac{c_2}{n^2} + \dots \right) \quad \text{für } n \rightarrow \infty$$

genau dann beschleunigt, wenn θ eine negative ganze Zahl ist. Außerdem zeigen wir, dass der ρ -Algorithmus keine lineare Konvergenz beschleunigen kann.

Definition 3.1 (Assoziierte Folge). Um das asymptotische Verhalten des ρ -Algorithmus zu beschreiben, verwenden wir eine verwandte Folge. Für gegebenes $\theta \notin \{0, 1\}$ und gegebenes $c \neq 0$ definieren wir eine Folge $(C_k)_{k=-1,0,1,\dots}$ durch:

$$C_{-1} = 0$$

$$C_0 = c$$

$$C_{2k-1} = C_{2k-3} + \frac{2k-1}{\theta C_{2k-2}}, \quad k = 1, 2, \dots$$

$$C_{2k} = C_{2k-2} + \frac{2k}{(1-\theta)C_{2k-1}}, \quad k = 1, 2, \dots$$

Diese Folge (C_k) nennen wir die assoziierte Folge des ρ -Algorithmus bezüglich θ und c .

Lemma 3.2. Die assoziierte Folge kann explizit dargestellt werden als

$$C_{2k-1} = \begin{cases} \frac{1}{c\theta}, & k = 1, \\ \frac{k(2-\theta)\cdots(k-\theta)}{c\theta(1+\theta)\cdots(k-1+\theta)}, & k > 1, \end{cases}$$

$$C_{2k} = \frac{c(1+\theta)\cdots(k+\theta)}{(1-\theta)\cdots(k-\theta)},$$

wobei

$$k \in \begin{cases} k \in \mathbb{N}, & \theta \notin \mathbb{Z}, \\ k < \theta, & \theta \in \mathbb{N}, \\ k \leq |\theta|, & -\theta \in \mathbb{N}. \end{cases}$$

Beweis. Der Beweis erfolgt durch Induktion. Die Darstellungen sind wahr für $k = 1$, weil

$$C_1 = C_{-1} + \frac{1}{c\theta} = \frac{1}{c\theta}, \quad \text{und} \quad C_2 = C_0 + \frac{2}{(1-\theta)C_1} = \frac{c(1+\theta)}{1-\theta}.$$

Nehmen wir an, dass sie für $k > 1$ wahr sind. Durch die Induktionsannahme erhalten wir

$$\begin{aligned} C_{2k+1} &= C_{2k-1} + \frac{2k+1}{\theta C_{2k}} \\ &= \frac{k(2-\theta) \cdots (k-\theta)}{c\theta(1+\theta) \cdots (k+\theta)} + \frac{(2k+1)(1-\theta) \cdots (k-\theta)}{c\theta(1+\theta) \cdots (k-1+\theta)} \\ &= \frac{(k+1)(2-\theta) \cdots (k-\theta)(k+1-\theta)}{c\theta(1+\theta) \cdots (k+\theta)}. \end{aligned}$$

Ebenso gilt

$$C_{2k+2} = \frac{c(1+\theta)(2+\theta) \cdots (k+1+\theta)}{(1-\theta) \cdots (k+1-\theta)}.$$

□

Satz 3.3. Sei (s_n) eine Folge mit

$$s_n \sim s + n^\theta \left(c_0 + \frac{c_1}{n} + \frac{c_2}{n^2} + \cdots \right), \quad n \rightarrow \infty$$

mit $\operatorname{Re} \theta < 0$, $c_0 \neq 0$ und Konstanten c_k . Sei (C_k) die assoziierte Folge zu θ und c_0 , und

$$A = (1-\theta) \left(-\frac{1}{2} + \frac{c_1}{c_0\theta} \right).$$

Dann gilt für festes $k \in \mathbb{N}$ mit $\theta \neq -1, \dots, 1-k$:

$$\begin{aligned} \rho_{2k-1}^{(n)} &= C_{2k-1}(n+k)^{1-\theta} \left(1 + \frac{A}{n+k} \right) + O((n+k)^{-1-\theta}), \\ \rho_{2k}^{(n)} &= s + C_{2k}(n+k)^\theta \left(1 + \frac{c_1}{c_0(n+k)} \right) + O((n+k)^{\theta-2}). \end{aligned}$$

Beweis. Wir führen eine vollständige Induktion über k .

Induktionsanfang ($k = 1$): Für $\rho_1^{(n)}$ und $\rho_2^{(n)}$ verwenden wir die Definition des ρ -Algorithmus:

$$\rho_1^{(n)} = n + \frac{1}{s_{n+1} - s_n}, \quad \rho_2^{(n)} = s_{n+1} + \frac{2}{(s_{n+2} - s_{n+1})^{-1} - (s_{n+1} - s_n)^{-1}}.$$

Mit der asymptotischen Entwicklung von s_n und binomischer Entwicklung erhalten wir:

$$s_{n+1} - s_n = c_0 \theta (n+1)^{\theta-1} \left[1 - \frac{A}{n+1} + \kappa_1 (n+1)^{-2} + O((n+1)^{\theta-3}) \right],$$

wobei $\kappa_1 = -\frac{(1-\theta)(2-\theta)}{12} + \frac{c_1(1-\theta)}{2c_0} + \frac{c_2}{c_0\theta}$. Einsetzen in $\rho_1^{(n)}$ liefert:

$$\rho_1^{(n)} = \frac{1}{c_0\theta} (n+1)^{1-\theta} \left[1 + \frac{A}{n+1} + \kappa_2 (n+1)^{-2} + O((n+1)^{-3}) \right] + n,$$

mit $\kappa_2 = A^2 - \kappa_1$. Da $C_1 = \frac{1}{c_0\theta}$ (aus [Lemma 3.2](#)) und $(n+1)^{1-\theta}$ dominiert n für $\operatorname{Re} \theta < 0$, folgt:

$$\rho_1^{(n)} = C_1 (n+1)^{1-\theta} \left(1 + \frac{A}{n+1} \right) + O((n+1)^{-1-\theta}).$$

Analog zeigt man für $\rho_2^{(n)}$:

$$\rho_2^{(n)} = s + C_2 (n+1)^\theta \left(1 + \frac{c_1}{c_0(n+1)} \right) + O((n+1)^{\theta-2}),$$

wobei $C_2 = \frac{c_0(1+\theta)}{1-\theta}$ ([Lemma 3.2](#)).

Induktionsschritt ($k \rightarrow k+1$): Annahme: Die Aussagen gelten für ein beliebiges $k \geq 1$.

1. Zeige $\rho_{2k+1}^{(n)}$:

Nach Definition des ρ -Algorithmus gilt:

$$\rho_{2k+1}^{(n)} = \rho_{2k-1}^{(n+1)} + \frac{2k+1}{\rho_{2k}^{(n+1)} - \rho_{2k}^{(n)}}.$$

Aus der Induktionsannahme für $\rho_{2k}^{(m)}$ folgt:

$$\rho_{2k}^{(n+1)} - \rho_{2k}^{(n)} = \theta C_{2k} (n+k+1)^{\theta-1} \left[1 - \frac{A}{n+k+1} \right] + O((n+k+1)^{\theta-3}).$$

Für $\rho_{2k-1}^{(n+1)}$ gilt:

$$\rho_{2k-1}^{(n+1)} = C_{2k-1}(n+k+1)^{1-\theta} \left(1 + \frac{A}{n+k+1} \right) + O((n+k+1)^{-1-\theta}).$$

Einsetzen und Umformen ergibt:

$$\begin{aligned} \rho_{2k+1}^{(n)} &= \underbrace{C_{2k-1} + \frac{2k+1}{\theta C_{2k}}}_{=C_{2k+1} \text{ (Rekursion aus Lemma 3.2)}} (n+k+1)^{1-\theta} \left(1 + \frac{A}{n+k+1} \right) + R_n, \end{aligned}$$

wobei der Restterm $R_n = O((n+k+1)^{-1-\theta})$ alle Fehler absorbiert. Somit:

$$\rho_{2k+1}^{(n)} = C_{2k+1}(n+k+1)^{1-\theta} \left(1 + \frac{A}{n+k+1} \right) + O((n+k+1)^{-1-\theta}).$$

2. Zeige $\rho_{2k+2}^{(n)}$:
Analog gilt:

$$\rho_{2k+2}^{(n)} = \rho_{2k}^{(n+1)} + \frac{2k+2}{\rho_{2k+1}^{(n+1)} - \rho_{2k+1}^{(n)}}.$$

Aus dem soeben Gezeigten folgt:

$$\rho_{2k+1}^{(n+1)} - \rho_{2k+1}^{(n)} = (1-\theta)C_{2k+1}(n+k+1)^{-\theta} \left[1 - \frac{c_1}{c_0(n+k+1)} \right] + O((n+k+1)^{-2-\theta}).$$

Mit Induktionsannahme für $\rho_{2k}^{(n+1)}$ folgt:

$$\rho_{2k}^{(n+1)} = s + C_{2k}(n+k+1)^\theta \left(1 + \frac{c_1}{c_0(n+k+1)} \right) + O((n+k+1)^{\theta-2}).$$

Einsetzen und Ausnutzen der Rekursion $C_{2k+2} = C_{2k} + \frac{2k+2}{(1-\theta)C_{2k+1}}$ (Lemma 3.2) liefert:

$$\rho_{2k+2}^{(n)} = s + C_{2k+2}(n+k+1)^\theta \left(1 + \frac{c_1}{c_0(n+k+1)} \right) + O((n+k+1)^{\theta-2}).$$

□

Satz 3.4 (Beschleunigung). Sei (s_n) eine Folge mit

$$s_n \sim s + n^\theta \left(c_0 + \frac{c_1}{n} + \frac{c_2}{n^2} + \dots \right), \quad \text{für } n \rightarrow \infty$$

mit $c_0 \neq 0$. Der ρ -Algorithmus beschleunigt die Konvergenz der Folge (s_n) genau dann, wenn θ eine negative ganze Zahl ist.

Beweis. Nach [Satz 3.3](#) gilt für jedes feste k :

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\rho_{2k}^{(n)} - s}{s_{n+2k} - s} &= \lim_{n \rightarrow \infty} \frac{s + C_{2k}(n+k)^\theta \left(1 + \frac{c_0}{c_1(n+k)}\right) + O((n+k)^{\theta-2}) - s}{s + (n+2k)^\theta \left(c_0 + \frac{c_1}{n+2k} + \frac{c_2}{(n+2k)^2} + \dots\right) - s} \\ &= \lim_{n \rightarrow \infty} \frac{C_{2k}(n+k)^\theta \left(1 + \frac{c_0}{c_1(n+k)}\right) + O((n+k)^{\theta-2})}{(n+2k)^\theta c_0 \left(1 + \frac{c_1}{c_0(n+2k)} + \frac{c_2}{c_0(n+2k)^2} + \dots\right)} \\ &= \frac{C_{2k}}{c_0} \end{aligned}$$

Im Normalfall ist $\frac{C_{2k}}{c_0} \neq 0$ und der Algorithmus konvergiert nicht schneller als die Originalfolge. Nach [Lemma 3.2](#) wissen wir aber:

$$\theta = -k \text{ für } k \in \mathbb{N} \implies C_0 \neq 0, \dots, C_{2k-2} \neq 0 \text{ und } C_{2k} = 0$$

Daraus folgt also direkt, dass $\lim_{n \rightarrow \infty} \frac{\rho_{2k}^{(n)} - s}{s_{n+2k} - s} = \frac{C_{2k}}{c_0} = 0$. Damit beschleunigt der ρ -Algorithmus die Folge s_n . □

3.5 Schwächen

Für die folgenden beiden Sätze beziehen wir uns auf [Osada, 1996, S. 526–527]. Sie werden grundlegend nur auf die Definition (3.6) angewandt (Zähler = $k+1$).

Satz 3.5. Wir verwenden hier eine alternative Definition für lineare Konvergenz. Sei (s_n) eine gegen s konvergente Folge mit $e_n = s_n - s$, wobei gilt:

$$e_{n+1} = (\lambda + \varepsilon_n)e_n, \quad \lambda \in [-1, 1) \setminus \{0\}, \varepsilon_n \rightarrow 0.$$

Dann gilt:

$$\rho_{2k}^{(n)} = s + (-1)^k e_{n+k} + O(e_{n+k} \varepsilon_{n,k-1}),$$

mit $\varepsilon_{n,k} = \max\{|\varepsilon_n|, \dots, |\varepsilon_{n+k}|\}$.

Beweis. Der Beweis dazu findet sich in [Osada, 1996, S. 526–527]. Die Idee des Beweises ist, zunächst eine rekursive, alternierende Folge D_k zu definieren und anschließend mittels Induktion zu zeigen, dass der ρ -Algorithmus in Abhängigkeit von dieser Folge dargestellt werden kann. □

Satz 3.6. Der ρ -Algorithmus kann lineare Konvergenz nicht beschleunigen.

Beweis. Nach [Satz 3.5](#) ergibt sich:

$$\begin{aligned}
\lim_{n \rightarrow \infty} \frac{\rho_{2k}^{(n)} - s}{s_{n+2k} - s} &= \lim_{n \rightarrow \infty} \frac{s + (-1)^k e_{n+k} + O(e_{n+k} \varepsilon_{n,k-1}) - s}{s + e_{n+2k} - s} \\
&= \lim_{n \rightarrow \infty} \frac{(-1)^k e_{n+k} + O(e_{n+k} \varepsilon_{n,k-1})}{e_{n+2k}} \\
&= \frac{(-1)^k \lambda^k e_n}{\lambda^{2k} e_n} \\
&= \frac{(-1)^k}{\lambda^k} \neq 0.
\end{aligned}$$

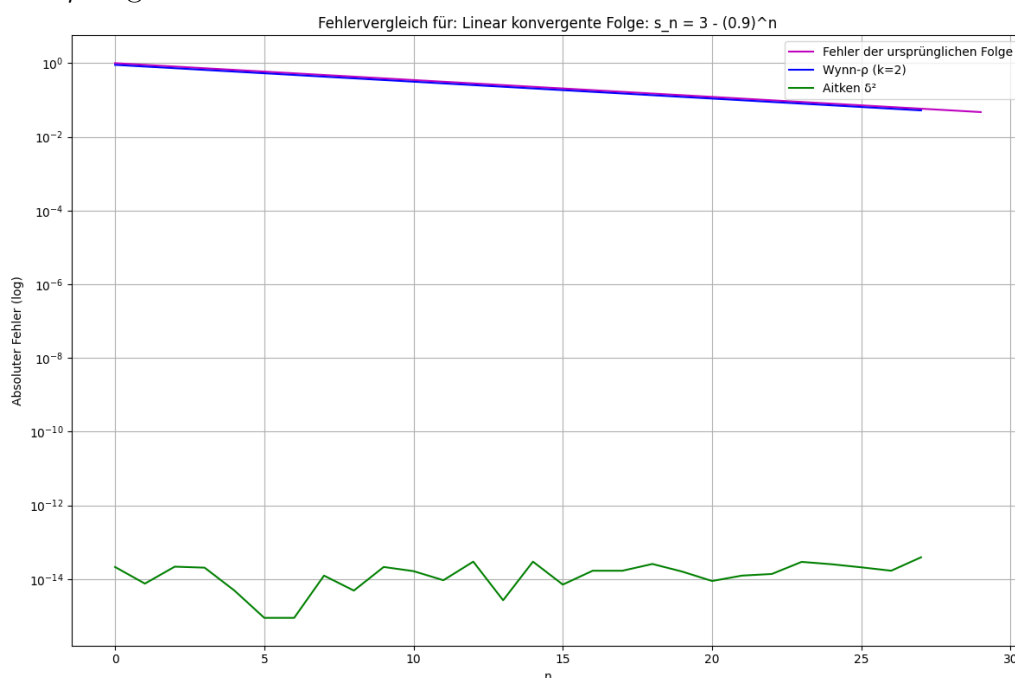
Damit wird keine Beschleunigung erzielt.

□

4 Beispiele, Tests & Vergleiche mit anderen Verfahren

4.1 Linear konvergente Folgen: Vergleich von ρ und Aitken- Δ^2

Folgende Aussage zum Aitken- Δ^2 -Verfahren finden wir in [Pomeranz, 2017, S. 2]. In [Abschnitt 3.5](#) wurde bewiesen, dass der ρ -Algorithmus zu keiner Beschleunigung bei linear konvergenten Folgen führt. Das kann man an dieser Grafik auch beispielhaft nachvollziehen. Wir vergleichen hier das Aitken- Δ^2 -Verfahren, welches linear konvergente Folgen sehr effizient beschleunigt, mit dem ρ -Algorithmus:

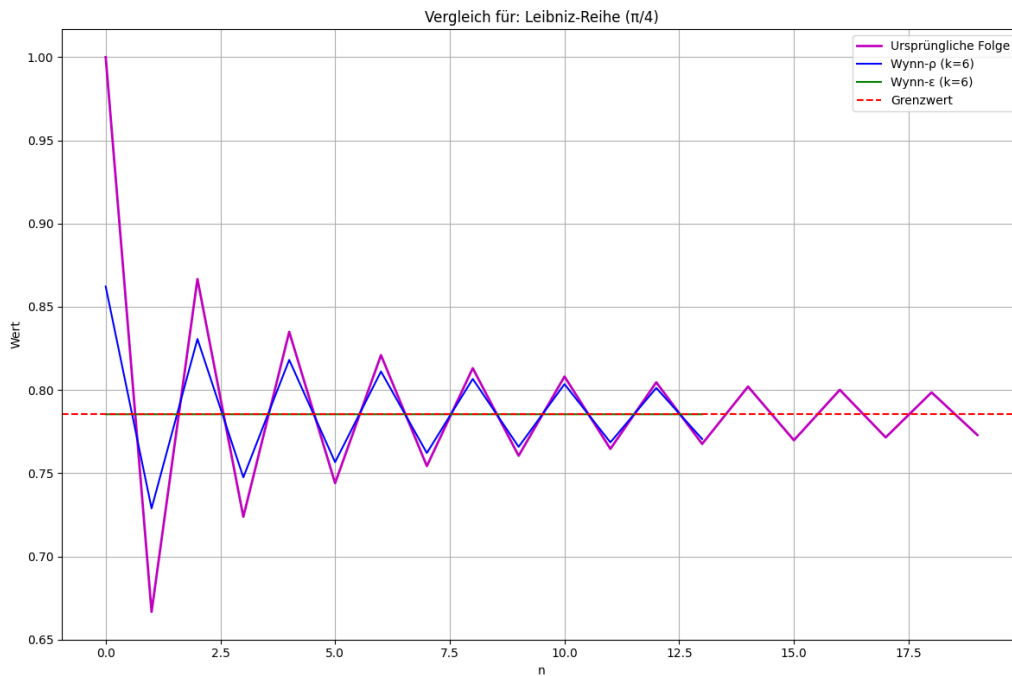


Während der Fehler beim Aitken- Δ^2 -Verfahren sehr schnell sehr klein ist, verhält sich der Fehler der ρ -Folge genau so wie der Fehler der originalen Folge s_n .

4.2 Alternierende Folgen: Vergleich von ρ und ϵ

Die Stärken und Schwächen des ϵ -Algorithmus werden in [Wynn, 1966, S. 112–115] genauer erläutert. Der ϵ -Algorithmus ist besonders effizient bei der Beschleunigung von alternierenden Folgen. Der ρ -Algorithmus führt hierbei zu kaum einer Verbesserung. Wir betrachten dafür die Leibniz-Reihe und vergleichen den ϵ -Algorithmus mit dem ρ -Algorithmus:

$$\sum_{k=0}^{\infty} (-1)^k \frac{1}{2k+1} = \frac{\pi}{4}$$



Der ϵ -Algorithmus erzielt schon in $n = 0$ einen sehr genauen Wert, während der ρ -Algorithmus sich an die Originalfolge annähert.

4.3 Logarithmisch konvergente Folgen

Der ρ -Algorithmus zeigt seine besondere Stärke bei der Beschleunigung logarithmisch konvergenter Folgen. Wir betrachten dabei die sogenannte *Euler – Mascheroni – Konstante* welche beschrieben wird durch

$$\lim_{n \rightarrow \infty} \sum_{k=1}^n \frac{1}{k} - \ln(n) \approx 0,57721566$$

Wir verwenden hier keine Grafik, da wir ausschließlich die Genauigkeit des ρ -Algorithmus analysieren wollen. Für den Algorithmus ergeben sich dann folgende Werte:

n	$k = -1$	$k = 0$	$k = 2$	$k = 4$	$k = 6$
0	0.00000000	1.00000000	0. <u>57660033</u>	0. <u>57728341</u>	0. <u>57721575</u>
1	0.00000000	0.80685282	0. <u>57692144</u>	0. <u>57722282</u>	0. <u>57721567</u>
2	0.00000000	0.73472104	0. <u>57706845</u>	0. <u>57721759</u>	0. <u>57721566</u>
3	0.00000000	0.69703897	0. <u>57713320</u>	0. <u>57721638</u>	0. <u>57721566</u>
4	0.00000000	0.67389542	0. <u>57716520</u>	0. <u>57721598</u>	0. <u>57721566</u>
5	0.00000000	0.65824053	0. <u>57718265</u>	0. <u>57721582</u>	0. <u>57721566</u>
6	0.00000000	0.64694699	0. <u>57719292</u>	0. <u>57721575</u>	0. <u>57721566</u>
7	0.00000000	0.63841560	0. <u>57719935</u>	0. <u>57721572</u>	0. <u>57721566</u>
8	0.00000000	0.63174368	0. <u>57720357</u>	0. <u>57721570</u>	0. <u>57721566</u>
9	0.00000000	0.62638316	0. <u>57720646</u>	0. <u>57721569</u>	0. <u>57721566</u>
10	0.00000000	0.62198207	0. <u>57720850</u>	0. <u>57721568</u>	0. <u>57721566</u>

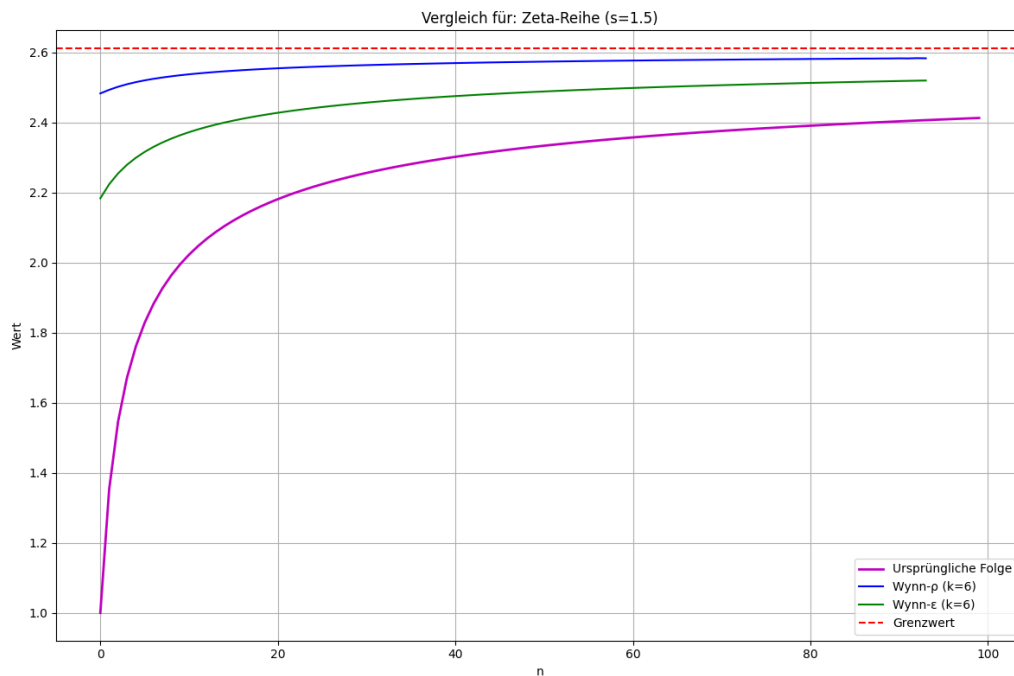
Wir erreichen dementsprechend das exakte Ergebnis bereits in Rekursionsschritt $k = 6$ mit Tiefe $n = 2$.

4.4 Langsam konvergente Folgen

Ebenso spannend gestaltet sich das Verhalten des ρ -Algorithmus bei Anwendung auf die Zeta-Funktion:

$$\zeta(s) = \sum_{n=1}^{\infty} \frac{1}{n^s}$$

Diese konvergiert besonders langsam, wenn $s \downarrow 1$. Dafür vergleichen wir den ρ -Algorithmus mit dem ϵ -Algorithmus, welcher hier große Schwierigkeiten hat. In der Grafik sei $s = 1,5$ der Beispielwert.



Es wird hier gut erkennbar, dass der ρ -Algorithmus im Vergleich noch sehr gut abschneidet, doch auch hier die Beschleunigung schon etwas schwerer gelingt. Bei noch stärkerer Annäherung an $s = 1$ nimmt die Effizienz des Algorithmus weiter ab. Diesem Problem wollen wir in [Abschnitt 4.5.1](#) entgegenwirken.

4.5 Modifikationen

Für die folgenden beiden Modifikationen beziehen wir uns auf [Sidi, 2003, S. 376–378], im folgenden Text finden sich jeweils nur die Definitionen mit einem Anwendungsbeispiel.

Durch eine kleine Veränderung der ursprünglichen Definition können wir in einigen Fällen sehr starke Verbesserungen erzielen. Im Wesentlichen verändern wir hier den Term, der in der klassischen Definition die Interpolationspunkte beschreibt (3.3), welche standardmäßig mit $(k + 1)$ definiert werden.

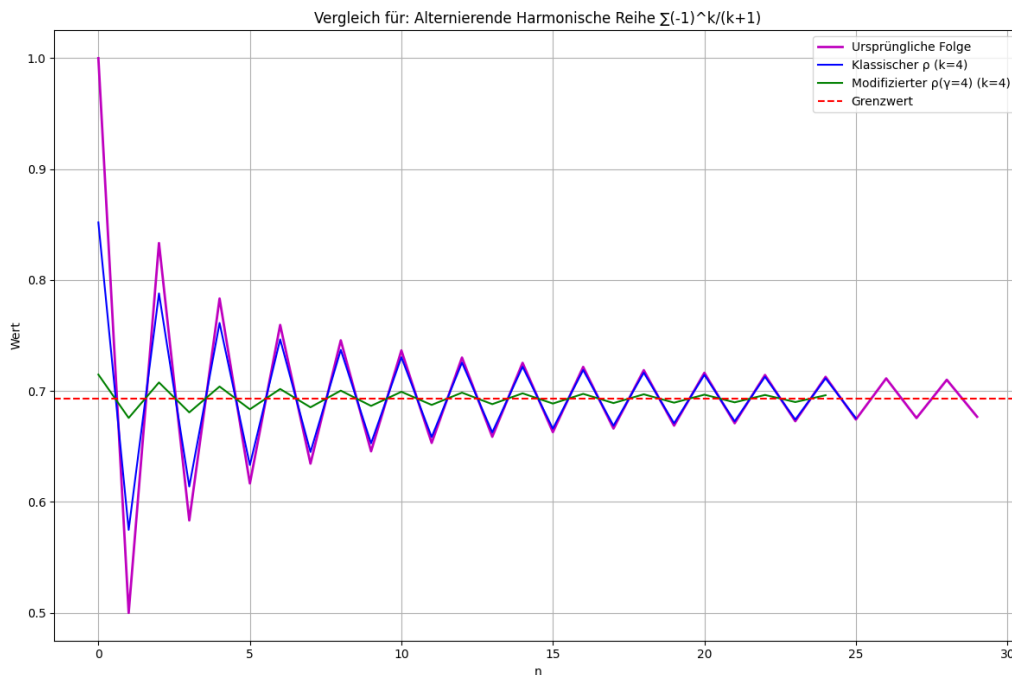
4.5.1 Modifikation 1 - $\rho(\gamma)$ -Algorithmus

Definition:

$$\begin{aligned}\bar{\rho}_{-1}^{(n)} &= 0 \\ \bar{\rho}_0^{(n)} &= s_n \\ \bar{\rho}_{k+1}^{(n)} &= \bar{\rho}_{k-1}^{(n+1)} + \frac{k - \gamma}{\bar{\rho}_k^{(n+1)} - \bar{\rho}_k^{(n)}}\end{aligned}$$

Leibniz-Reihe

Wenn wir diesen Algorithmus nun mit bestimmtem γ auf die *Leibniz-Reihe* anwenden, erzielen wir im Gegensatz zum klassischen Algorithmus eine starke Verbesserung der Konvergenz. Dazu betrachten wir das Beispiel der Alternierenden Harmonischen Reihe:



Wir erkennen hier eine sehr starke Verbesserung im Vergleich zum ursprünglichen Algorithmus. Wählen wir nun unser γ noch größer, nähern wir uns immer stärker dem tatsächlichen Grenzwert an.

Zeta-Funktion

In [Abschnitt 4.4](#) haben wir eine eingeschränkte Beschleunigung für die Zeta-Funktion durch den ρ -Algorithmus bereits festgestellt. Durch geeignete Wahl der Parameter können wir das jedoch optimieren.

Der exakte Wert ist

$$\zeta(1, 1) = 10,58444846$$

Während der klassische ρ -Algorithmus bei einer Rekursionstiefe von $n = 23$ noch einen Fehler von ($e_{23} > 5$) aufweist, kann unser modifizierter $\rho(\gamma)$ -Algorithmus mit dem Parameter $\gamma = 0,9$ einen Fehler von ($e_{23}^\gamma \approx 10^{-8}$) erzielen.

Hierbei ist der Parameter sehr sorgfältig zu wählen; kleine Abweichungen können in diesem Fall die Beschleunigung völlig ruinieren.

n	Original	ρ (standard)	$\rho(\gamma = 0, 9)$
0	1.00000000	4.05905983	<u>10.58439421</u>
1	1.46651650	4.21881639	<u>10.58442870</u>
2	1.76516932	4.34421820	<u>10.58443969</u>
3	1.98280696	4.44700181	<u>10.58444402</u>
4	2.15307494	4.53383717	<u>10.58444598</u>
5	2.29240141	4.60885546	<u>10.58444697</u>
6	2.40999730	4.67478432	<u>10.58444752</u>
7	2.51152885	4.73351524	<u>10.58444784</u>
8	2.60072236	4.78641165	<u>10.58444803</u>
9	2.68015518	4.83448760	<u>10.58444816</u>
10	2.75168186	4.87851702	<u>10.58444824</u>
11	2.81667995	4.91910356	<u>10.58444830</u>
12	2.87619987	4.95672683	<u>10.58444834</u>
13	2.93106029	4.99177397	<u>10.58444837</u>
14	2.98191131	5.02456188	<u>10.58444839</u>
...	...	...	...
23	3.32204995	5.25044529	<u>10.58444845</u>

4.5.2 Automatisierter $\rho(\gamma)$ -Algorithmus

Folgend soll noch ein kurzer Einblick in eine mögliche Automatisierung des $\rho(\gamma)$ -Verfahrens gegeben werden. Weitere Ausführungen zur Automatisierung und zur Verbindung zu anderen Beschleunigungsverfahren lassen sich in [Sidi, 2003, S. 378] nachlesen.

Der automatisierte $\rho(\gamma)$ -Algorithmus beschleunigt die Konvergenz von Folgen $\{s_n\}$ durch adaptive Parameterschätzung.

Die Idee ist, dass für Folgen mit asymptotischem Verhalten

$$s_n \sim s + \sum_{i=0}^{\infty} \alpha_i n^{\gamma-i} \quad (n \rightarrow \infty)$$

der Algorithmus automatisch den optimalen Parameter γ' bestimmt und den $\rho(\gamma')$ -Algorithmus anwendet.

Algorithmus

1. Parameterschätzung:

- Eingabe: $s_0, \dots, s_r$
- Wende $\rho(-2)$ -Algorithmus auf Hilfsfolge $\{\gamma_m\}_{m=0}^{r-2}$ an

- Setze:

$$\gamma' = \begin{cases} \bar{\rho}_{2k}^{(0)} & \text{für } r = 2k + 2 \\ \bar{\rho}_{2k}^{(1)} & \text{für } r = 2k + 3 \end{cases}$$

2. Konvergenzbeschleunigung:

- Wende $\rho(\gamma')$ -Algorithmus auf $\{s_n\}$ an
- Ausgabe: Beschleunigte Näherungen $\bar{\rho}_{2k}^{(j)}$ für $0 \leq j + 2k \leq r$

Der große Vorteil liegt hier darin, dass keine Schätzung des γ -Parameters nötig ist.

4.5.3 Modifikation 2 - $\rho(\gamma, p)$ -Algorithmus

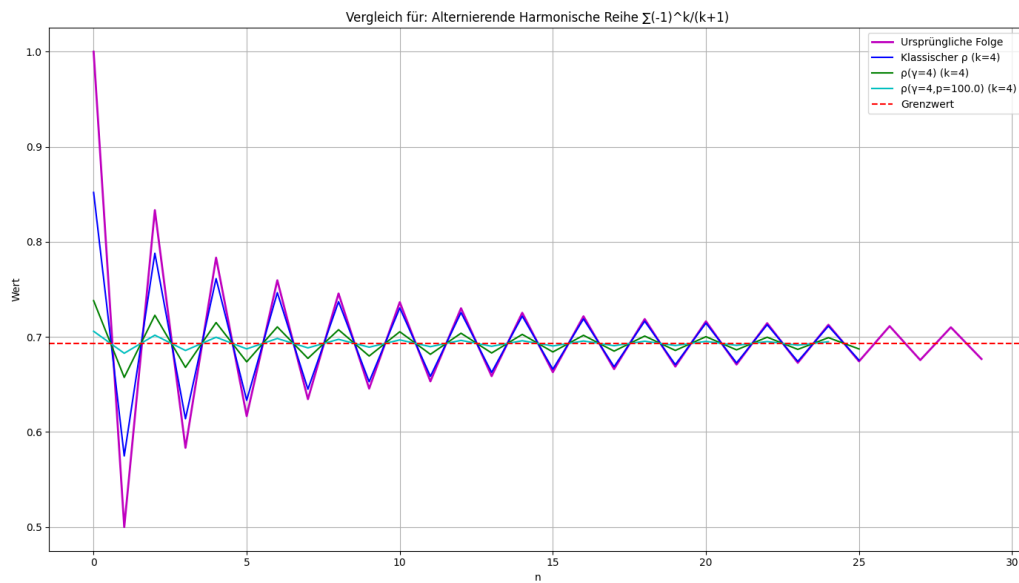
Definition:

$$\begin{aligned} \hat{\rho}_{-1}^{(n)} &= 0 \\ \hat{\rho}_0^{(n)} &= s_n \\ \hat{\rho}_{k+1}^{(n)} &= \hat{\rho}_{k-1}^{(n+1)} + \frac{C_k^{(\gamma, p)}}{\hat{\rho}_k^{(n+1)} - \hat{\rho}_k^{(n)}} \end{aligned}$$

wobei

$$C_{2k}^{(\gamma, p)} = -\gamma + \frac{k}{p}, \quad C_{2k+1}^{(\gamma, p)} = -\gamma + \frac{k}{p} + 1$$

Die Abweichung zu Modifikation 1 bezieht sich auf den Zähler der 3. Gleichung. Der Schritt k wird skaliert um $\frac{1}{p}$. Das führt unter Umständen zu einer weiteren kleinen Verbesserung im Vergleich zu Modifikation 1. Dazu betrachten wir das Beispiel der Alternierenden Harmonischen Reihe:



Es zeigt sich in den Anwendungen, dass sich eine grobe Verbesserung durch das Verändern des γ -Parameters erzielen lässt, der Parameter p bezweckt hier nur eine minimale Verbesserung um $\approx 10^{-6}$.

Literaturverzeichnis

- [1] Naoki Osada. “An acceleration theorem for the ρ -algorithm”. In: *Numerische Mathematik* 73 (1996), S. 521–531.
- [2] Shirley B Pomeranz. “Aitken’s Δ^2 method extended”. In: *Cogent Mathematics* 4.1 (2017), S. 1308622.
- [3] Avram Sidi. *Practical extrapolation methods: Theory and applications*. Bd. 10. Cambridge university press, 2003.
- [4] Ernst Joachim Weniger. “Nonlinear sequence transformations for the acceleration of convergence and the summation of divergent series”. In: *Computer Physics Reports* 10.5-6 (1989), S. 189–371.
- [5] Peter Wynn. “On the convergence and stability of the epsilon algorithm”. In: *SIAM Journal on Numerical Analysis* 3.1 (1966), S. 91–122.